Chapter 5
Unleashing the power of Transfer Learning

Topics

1. Need for TL
2. Building CNN models from scratch
    1. Basic CNN Model
    2. Improving CNN Model with regularization
    3. Improving CNN Model with Augmentation
3. Leveraging TL with pretrained CNN Models
    1. Pretrained Model as Feature Extractor
    2. Improving Pretrained Model with Augmentation
    3. Improving Pretrained Model with fine-tuning and Augmentation
4. Model Performance Evaluation

1. Need for TL

- TL offers several advantages like:
  - Improved baseline performance
  - Model development time will be less
  - Improved final performance.
- We need lot of training data to build robust deep learning models. The model then can learn features automatically from a large number of samples.
- What if we don't have enough samples and the problem to solve is a complex problem.
- Even if we have image samples ranging from a few hundred to thousands per category, a basic CNN model will still perform decently with right architecture and regularization
- With regard to TL, there are some excellent pretrained models that have been trained on the famous ImageNet dataset.
- The idea is to use a pretrained model that is usually an expert in the problem given with the constraint of fewer data samples

2.Building CNN models from scratch:
2.1Basic CNN Model

```
# Importing necessary libraries
import tensorflow as tf
from tensorflow import keras
from keras.utils import image_dataset_from_directory
import matplotlib.pyplot as plt
# Sets Path to train and test Data directories
train_dir = '/content/drive/My Drive/DogvsCat/train'
test_dir = '/content/drive/My Drive/DogvsCat/test‘

# creates a training  and testing dataset from the images in the train_dir and test_dir  directory.
# The images are resized to  64x64 pixels, and the data is batched into groups of 32 images.
train_generator = image_dataset_from_directory(train_dir, image_size=(64, 64), batch_size=32)
test_generator = image_dataset_from_directory(test_dir, image_size=(64, 64), batch_size=32)
```

The image_dataset_from_directory function simplifies the process of loading and preprocessing image data by creating TensorFlow datasets directly from directories of images.

```
# Building the CNN model
#Initializes a sequential model in Keras. The Sequential class is used to create a linear
#stack of layers, which means you can easily add one layer at a time
model = keras.Sequential([
    # Conv layer 1:
    keras.layers.Conv2D(128, (3, 3), input_shape=(64, 64, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    # Conv layer 2:
    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    # Conv layer 3:
    keras.layers.Conv2D(32, (3, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Flatten(),
    # Fully connected layers:
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dense(units=1, activation='sigmoid')
])
```

this code defines a CNN model with three convolutional layers, each followed by batch normalization and max pooling layers. After the convolutional layers, a flattening layer is added to convert the 2D feature maps into a 1D vector, which is then passed through two fully connected layers. The final output layer uses a sigmoid activation function, making the model suitable for binary classification.

keras.layers.Conv2D(128, (3, 3), input_shape=(64, 64, 3), activation='relu'), adds a 2D convolutional layer with 128 filters, each of size 3x3. The input_shape=(64, 64, 3) specifies that the input to this layer is 64x64 images with 3 color channels (RGB).

The activation='relu' applies the ReLU activation function, which introduces non-linearity to the model.

keras.layers.BatchNormalization(): This normalizes the output of the previous layer, which can help speed up training and improve the model's performance.

keras.layers.MaxPooling2D(pool_size=(2, 2)): This layer performs max pooling with a 2x2 pool size, which reduces the spatial dimensions (height and width) of the output volume, helping to down-sample the representation and reduce the computational load

keras.layers.Flatten(): This layer flattens the input, converting the 2D matrix data to a 1D vector. This is necessary before feeding the data into fully connected (dense) layers.

keras.layers.Dense(units=128, activation='relu'): This adds a fully connected (dense) layer with 128 units (neurons). The ReLU activation function is applied.

```
# Compile the model
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss='binary_crossentropy',
metrics=['accuracy'])
 # Print the model summary
model.summary()
 # Train the model
```

model.compile(...): This method configures the model for training.
optimizer=keras.optimizers.Adam(learning_rate=0.001): Specifies the optimizer to use for training. In this case, it is the Adam optimizer with a learning rate of 0.001. Adam is a popular choice due to its adaptive learning rate properties.
loss='binary_crossentropy': Specifies the loss function to use. Binary crossentropy is used here because the task is binary classification (dog vs cat).
metrics=['accuracy']: Specifies the metrics to evaluate during training and testing. Here, accuracy is used to measure the performance of the model.
model.summary(): This method prints a summary of the model's architecture. It provides details about each layer, including the type, output shape, and number of parameters.

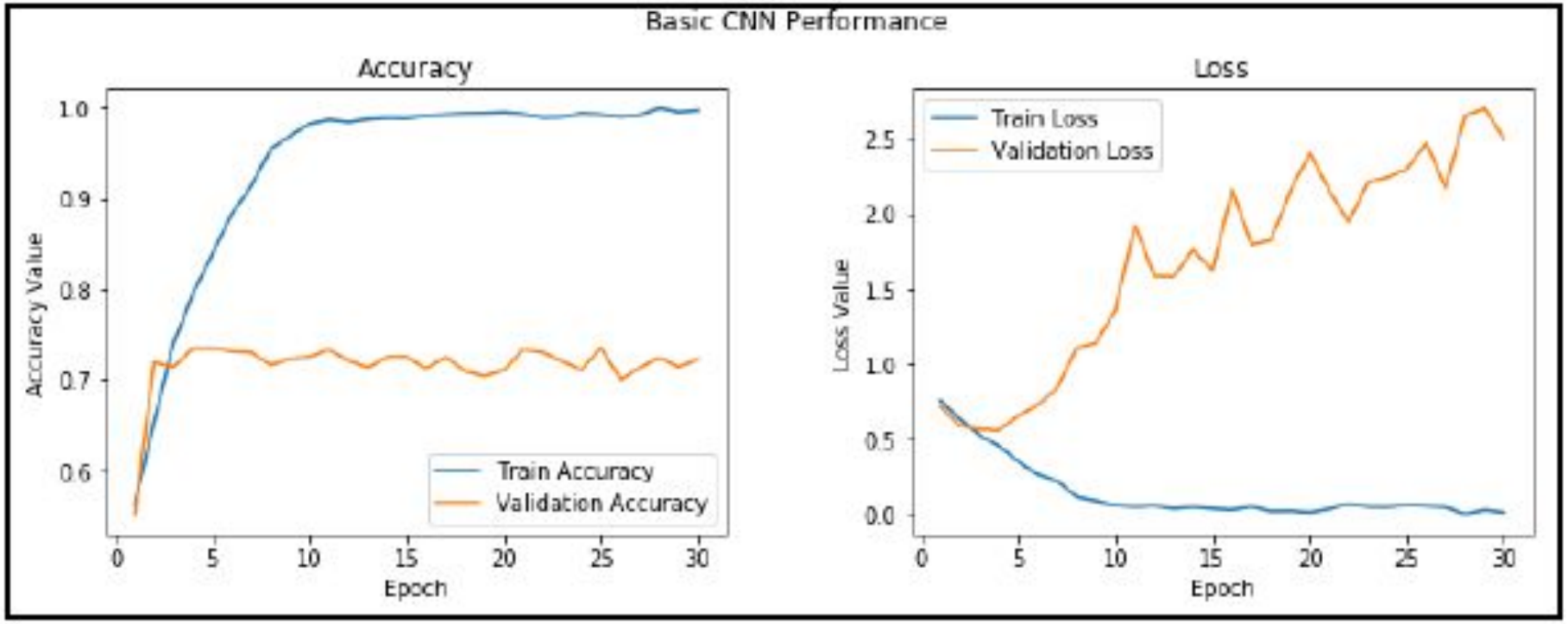
```
logs = model.fit(train_generator, epochs=10, validation_data=test_generator)
 # Plotting the training log
plt.figure(figsize=(10, 5))
plt.title('Training Log')
plt.plot(logs.history['loss'], label='Training Loss')
plt.plot(logs.history['accuracy'], label='Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Score')
plt.legend()
plt.show()
 # Evaluate the model
res = model.evaluate(test_generator)
accuracy = res[1]
print("Test Accuracy:", accuracy)
```

model.fit(...): This method trains the model for a fixed number of epochs (iterations over the dataset).

- train_generator: The training data. This is an instance of a data generator that provides batches of training data.
- epochs=10: Specifies the number of epochs to train the model. An epoch is one complete pass through the training dataset.
- validation_data=test_generator: Specifies the validation data. This is an instance of a data generator that provides batches of validation data. The model's performance on this data will be evaluated at the end of each epoch.

The model.fit() method returns a History object, which contains a record of training loss and metrics values at successive epochs, as well as validation loss and metrics values

model.evaluate(test_generator): This method evaluates the model on the test data provided by the test_generator. The evaluate method returns a list of metrics values (e.g., loss and accuracy) for the model on the test dataset. The values correspond to the metrics specified during the compile step.


Basic CNN Performance
Accuracy
Accuracy Value
Epoch
Train Accuracy
Validation Accuracy
Loss
Loss Value
Epoch
Train Loss
Validation Loss

2.2 Improving CNN Model with regularization

```
import tensorflow as tf
from tensorflow import keras
from keras.utils import image_dataset_from_directory
import matplotlib.pyplot as plt

# Data directories
train_dir = '/content/drive/My Drive/DogvsCat/train'
test_dir = '/content/drive/My Drive/DogvsCat/test'

# Creating datasets (in form of image generators)
train_generator = image_dataset_from_directory(train_dir, image_size=(64, 64), batch_size=32)
test_generator = image_dataset_from_directory(test_dir, image_size=(64, 64), batch_size=32)
```

2.2 Improving CNN Model with regularization

```
# Building the improved CNN model with regularization
model = keras.Sequential([
    # Conv layer 1:
    keras.layers.Conv2D(128, (3, 3), input_shape=(64, 64, 3), activation='relu', kernel_regularizer=keras.regularizers.l2(0.001)),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Dropout(0.3),
    # Conv layer 2:
    keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_regularizer=keras.regularizers.l2(0.001)),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Dropout(0.3),
    # Conv layer 3:
    keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_regularizer=keras.regularizers.l2(0.001)),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Dropout(0.3),
    keras.layers.Flatten(),
    # Fully connected layers:
    keras.layers.Dense(units=128, activation='relu', kernel_regularizer=keras.regularizers.l2(0.001)),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=1, activation='sigmoid')
])
```

keras.regularizers.l2(0.001): This creates an L2 regularizer with a penalty term coefficient of 0.001. This means that the sum of the squared weights is multiplied by 0.001 and added to the loss function. This encourages the model to keep the weights small, which can help improve generalization and prevent overfitting.

Dropout(0.3):

This is the dropout rate, which represents the fraction of the input units to drop. In this case, it is 0.3, meaning that 30% of the input units will be randomly set to 0 during each training step. This forces the network to learn to spread out its weights and prevents it from relying too heavily on specific neurons.

2.3 Improving CNN Model with Augmentation

```
# Importing necessary libraries
import tensorflow as tf
from tensorflow import keras
from keras.utils import image_dataset_from_directory
import matplotlib.pyplot as plt

# Data directories
train_dir = '/content/drive/My Drive/DogvsCat/train'
test_dir = '/content/drive/My Drive/DogvsCat/test'

# Creating data augmentation generator
train_datagen = keras.preprocessing.image.ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

- keras.preprocessing.image.ImageDataGenerator: This class provides real-time data augmentation and will generate batches of tensor image data with real-time data augmentation. The data will be looped over (in batches) indefinitely.

Parameters:

- rescale=1./255: This rescales the pixel values from a range of [0, 255] to [0, 1] by dividing each pixel value by 255. This is a common preprocessing step to normalize the data.
- rotation_range=40: This randomly rotates the images within the range of 0 to 40 degrees. This helps the model learn to recognize images from different angles.
- width_shift_range=0.2: This randomly shifts the image horizontally by a fraction of the total width. The range is specified as a fraction of the total width, in this case, 20% (0.2). This helps the model learn to recognize objects that may not be centered.
- height_shift_range=0.2: This randomly shifts the image vertically by a fraction of the total height. Similar to width_shift_range, this is set to 20% (0.2).
- shear_range=0.2: This applies shear transformations, which can be thought of as a combination of translation and rotation. The shear angle is set to 20% (0.2).
- zoom_range=0.2: This randomly zooms in or out on the image by a factor of 20% (0.2). This helps the model learn to recognize objects at different scales.
- horizontal_flip=True: This randomly flips half of the images horizontally. This is useful when there is no inherent left-right asymmetry in the data (e.g., cats and dogs can appear in any direction).
- fill_mode='nearest': This specifies the strategy used for filling in newly created pixels after a transformation (such as a rotation or a shift). 'nearest' means that the new pixels will be filled with the nearest pixel values from the original image.

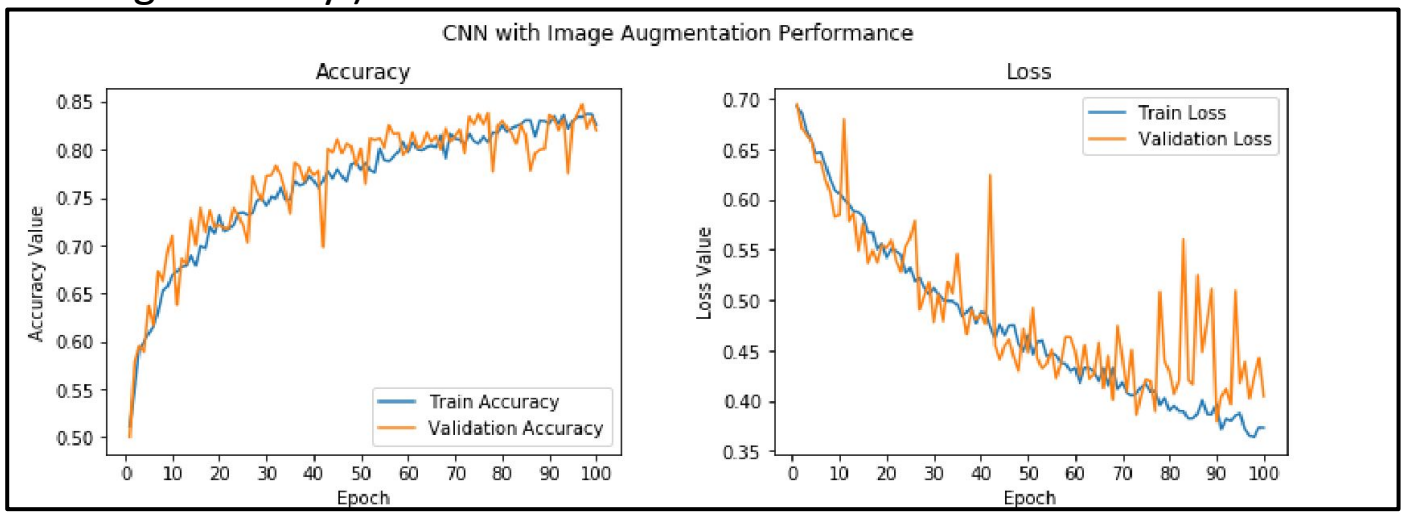
```
test_datagen =
keras.preprocessing.image.ImageDataGenerator(rescale=1./255)

# Creating datasets (in form of image generators)
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(64, 64),
    batch_size=32,
    class_mode='binary'
)
 test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(64, 64),
    batch_size=32,
    class_mode='binary'
)
```

train_generator = train_datagen.flow_from_directory(...):
train_datagen is an instance of ImageDataGenerator from Keras, which is used for real-time data augmentation and preprocessing of images.
flow_from_directory method generates batches of image data from a directory.
Arguments:

- train_dir: Path to the directory containing the training images organized into subdirectories by class labels.
- target_size=(64, 64): Resizes all images to a fixed size of 64x64 pixels.
- batch_size=32: Number of images to yield from the generator in each batch.
- class_mode='binary': Determines the type of label arrays that are returned. 'binary' means the generator will yield 1D numpy arrays of binary labels.

```
# Building the CNN model – same as in 2.1
model = keras.Sequential([
    # Conv layer 1:
    keras.layers.Conv2D(128, (3, 3), input_shape=(64, 64, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),

    # Conv layer 2:
    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),

    # Conv layer 3:
    keras.layers.Conv2D(32, (3, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Flatten(),

    # Fully connected layers:
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dense(units=1, activation='sigmoid')
])
```

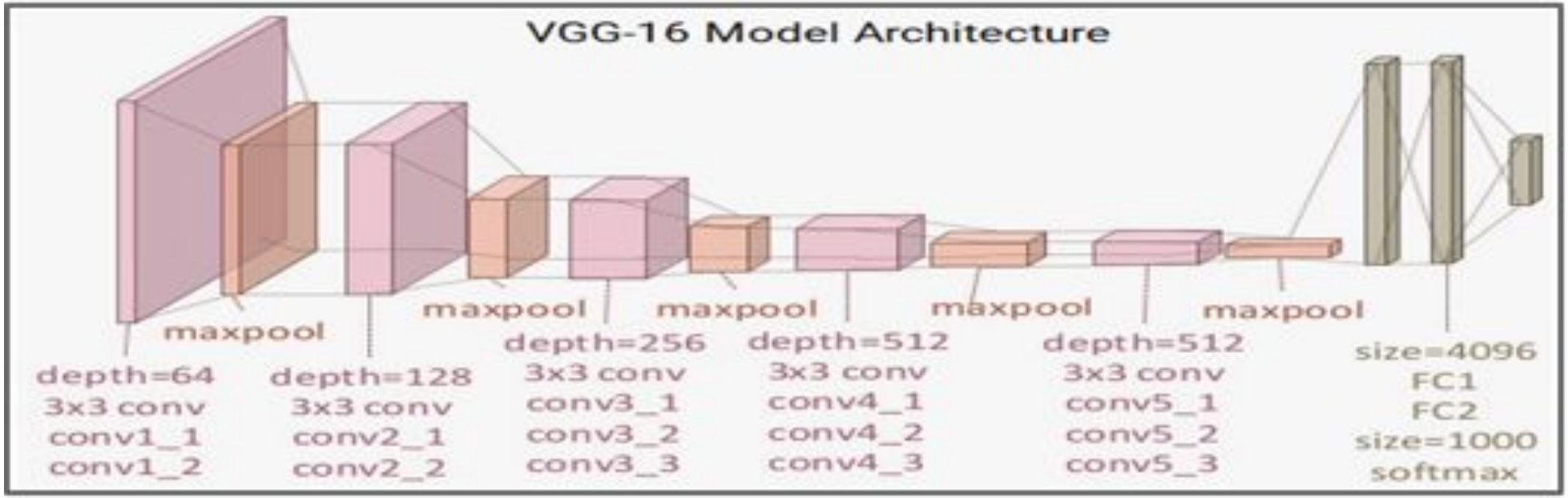
```
# Compile the model
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss='binary_crossentropy',
metrics=['accuracy'])
 # Print the model summary
model.summary()
 # Train the model
logs = model.fit(train_generator, epochs=10, validation_data=test_generator)
 # Plotting the training log
plt.figure(figsize=(10, 5))
plt.title('Training Log')
plt.plot(logs.history['loss'], label='Training Loss')
plt.plot(logs.history['accuracy'], label='Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Score')
plt.legend()
plt.show()
 # Evaluate the model
res = model.evaluate(test_generator)
accuracy = res[1]
print("Test Accuracy:", accuracy)
```



3. Leveraging TL with pretrained CNN Models

- Pretrained VGG16 model for feature extraction with a custom classifier on top for a binary classification task
- Pretrained CNN model as feature Extractor with Augmentation
- Pretrained Model with fine-tuning and image augmentation

VGG16 Model


VGG-16 Model Architecture
maxpool
maxpool
maxpool
maxpool
maxpool
depth=64
3x3 conv
conv1_1
conv1_2
depth=128
3x3 conv
conv2_1
conv2_2
depth=256
3x3 conv
conv3_1
conv3_2
conv3_3
depth=512
3x3 conv
conv4_1
conv4_2
conv4_3
depth=512
3x3 conv
conv5_1
conv5_2
conv5_3
size=4096
FC1
FC2
size=1000
softmax

3.1 pretrained VGG16 model for feature extraction with a custom classifier on top for a binary classification task

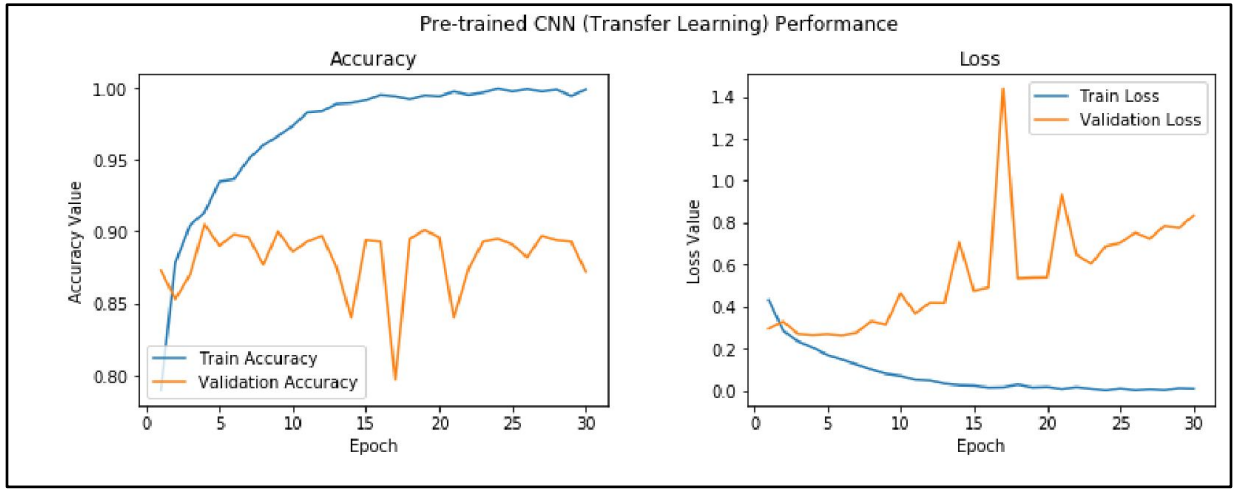
#Feature Extraction with VGG16

```
from keras.applications import vgg16
from keras.models import Model
import keras
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, InputLayer
from keras.models import Sequential
from keras import optimizers

vgg = vgg16.VGG16(include_top=False, weights='imagenet',   input_shape=input_shape)

output = vgg.layers[-1].output
output = keras.layers.Flatten()(output)

vgg_model = Model(vgg.input, output)
vgg_model.trainable = False

for layer in vgg_model.layers:
    layer.trainable = False

vgg_model.summary()
```

	Layer Type	Layer Name	Layer Trainable
0	<keras.engine.topology.InputLayer object at 0x7f26c86b2518>	input_1	False
1	<keras.layers.convolutional.Conv2D object at 0x7f277c9fc080>	block1_conv1	False
2	<keras.layers.convolutional.Conv2D object at 0x7f26c86b26d8>	block1_conv2	False
3	<keras.layers.pooling.MaxPooling2D object at 0x7f26c86e6c88>	block1_pool	False
4	<keras.layers.convolutional.Conv2D object at 0x7f26c867dc18>	block2_conv1	False
5	<keras.layers.convolutional.Conv2D object at 0x7f26c8690f28>	block2_conv2	False
6	<keras.layers.pooling.MaxPooling2D object at 0x7f26c869e5c0>	block2_pool	False
7	<keras.layers.convolutional.Conv2D object at 0x7f26c863f828>	block3_conv1	False
8	<keras.layers.convolutional.Conv2D object at 0x7f26c863f128>	block3_conv2	False
9	<keras.layers.convolutional.Conv2D object at 0x7f26c86607b8>	block3_conv3	False
10	<keras.layers.pooling.MaxPooling2D object at 0x7f26c83d7d68>	block3_pool	False
11	<keras.layers.convolutional.Conv2D object at 0x7f26c83fd358>	block4_conv1	False
12	<keras.layers.convolutional.Conv2D object at 0x7f26c83fddd8>	block4_conv2	False
13	<keras.layers.convolutional.Conv2D object at 0x7f26c839da20>	block4_conv3	False
14	<keras.layers.pooling.MaxPooling2D object at 0x7f26c83ac1d0>	block4_pool	False
15	<keras.layers.convolutional.Conv2D object at 0x7f26c834e978>	block5_conv1	False
16	<keras.layers.convolutional.Conv2D object at 0x7f271a15eb38>	block5_conv2	False
17	<keras.layers.convolutional.Conv2D object at 0x7f26c8371d68>	block5_conv3	False
18	<keras.layers.pooling.MaxPooling2D object at 0x7f26c8314b00>	block5_pool	False
19	<keras.layers.core.Flatten object at 0x7f26c828bda0>	flatten_1	False

#Custom Classifier on Top of VGG16

```
print("Trainable layers:", vgg_model.trainable_weights)
input_shape = vgg_model.output_shape[1]
model = Sequential()
model.add(InputLayer(input_shape=(input_shape,)))
model.add(Dense(512, activation='relu', input_dim=input_shape))
model.add(Dropout(0.3))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy',
          optimizer=optimizers.RMSprop(lr=1e-4),
          metrics=['accuracy'])
model.summary()
history = model.fit(x=train_features_vgg, y=train_labels_enc,
              validation_data=(validation_features_vgg, validation_labels_enc),
              batch_size=batch_size,
              epochs=epochs,
              verbose=1)
```



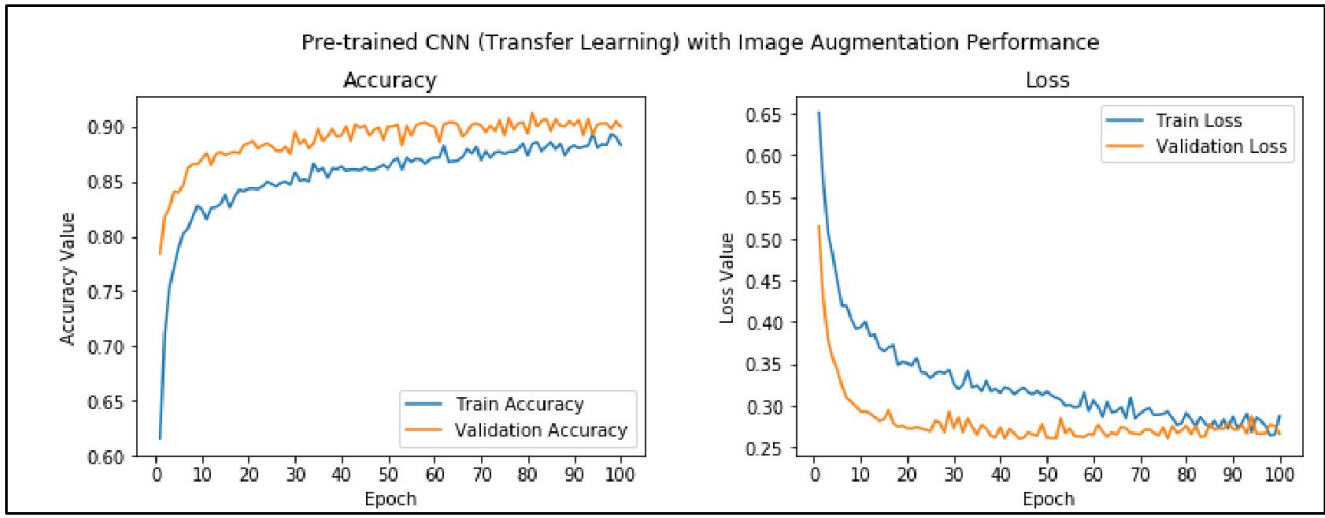
3.2 Pretrained CNN model as feature Extractor with Augmentation

```
from keras.applications import VGG16
from keras.models import Model, Sequential
from keras.layers import Flatten, Dense, Dropout, InputLayer
from keras.optimizers import RMSprop
from keras.preprocessing.image import ImageDataGenerator
 # Define input shape for your images and batch size
input_shape = (224, 224, 3)
batch_size = 32
# Load pretrained VGG16 model without the top (fully connected) layers
vgg = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
# Freeze all layers of the VGG16 base
for layer in vgg.layers:
    layer.trainable = False
# Data augmentation for training set
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)
```

```
# Data augmentation for validation set (only rescaling)
validation_datagen = ImageDataGenerator(rescale=1./255)

# Path to your training and validation data directories
train_dir = 'path_to_training_directory'
validation_dir = 'path_to_validation_directory'

# Flow training images in batches using train_datagen generator
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(input_shape[0], input_shape[1]),
    batch_size=batch_size,
    class_mode='binary'
)

# Flow validation images in batches using validation_datagen generator
validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=(input_shape[0], input_shape[1]),
    batch_size=batch_size,
    class_mode='binary'
)
```

	Layer Type	Layer Name	Layer Trainable
0	<keras.engine.topology.InputLayer object at 0x7f26c86b2518>	input_1	False
1	<keras.layers.convolutional.Conv2D object at 0x7f277c9fc080>	block1_conv1	False
2	<keras.layers.convolutional.Conv2D object at 0x7f26c86b26d8>	block1_conv2	False
3	<keras.layers.pooling.MaxPooling2D object at 0x7f26c86e6c88>	block1_pool	False
4	<keras.layers.convolutional.Conv2D object at 0x7f26c867dc18>	block2_conv1	False
5	<keras.layers.convolutional.Conv2D object at 0x7f26c8690f28>	block2_conv2	False
6	<keras.layers.pooling.MaxPooling2D object at 0x7f26c869e5c0>	block2_pool	False
7	<keras.layers.convolutional.Conv2D object at 0x7f26c863f828>	block3_conv1	False
8	<keras.layers.convolutional.Conv2D object at 0x7f26c863f128>	block3_conv2	False
9	<keras.layers.convolutional.Conv2D object at 0x7f26c86607b8>	block3_conv3	False
10	<keras.layers.pooling.MaxPooling2D object at 0x7f26c83d7d68>	block3_pool	False
11	<keras.layers.convolutional.Conv2D object at 0x7f26c83fd358>	block4_conv1	True
12	<keras.layers.convolutional.Conv2D object at 0x7f26c83fddd8>	block4_conv2	True
13	<keras.layers.convolutional.Conv2D object at 0x7f26c839da20>	block4_conv3	True
14	<keras.layers.pooling.MaxPooling2D object at 0x7f26c83ac1d0>	block4_pool	True
15	<keras.layers.convolutional.Conv2D object at 0x7f26c834e978>	block5_conv1	True
16	<keras.layers.convolutional.Conv2D object at 0x7f271a15eb38>	block5_conv2	True
17	<keras.layers.convolutional.Conv2D object at 0x7f26c8371d68>	block5_conv3	True
18	<keras.layers.pooling.MaxPooling2D object at 0x7f26c8314b00>	block5_pool	True
19	<keras.layers.core.Flatten object at 0x7f26c828bda0>	flatten_1	True

```
# Create a new model using VGG16 base and custom top layers for classification
model = Sequential()
model.add(vgg)
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))
# Compile the model with binary crossentropy loss and RMSprop optimizer
model.compile(loss='binary_crossentropy',
              optimizer=RMSprop(lr=1e-4),
              metrics=['accuracy'])
# Print model summary
model.summary()
# Train the model using data generators
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.n // batch_size,
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=validation_generator.n // batch_size,
    verbose=1
)
```

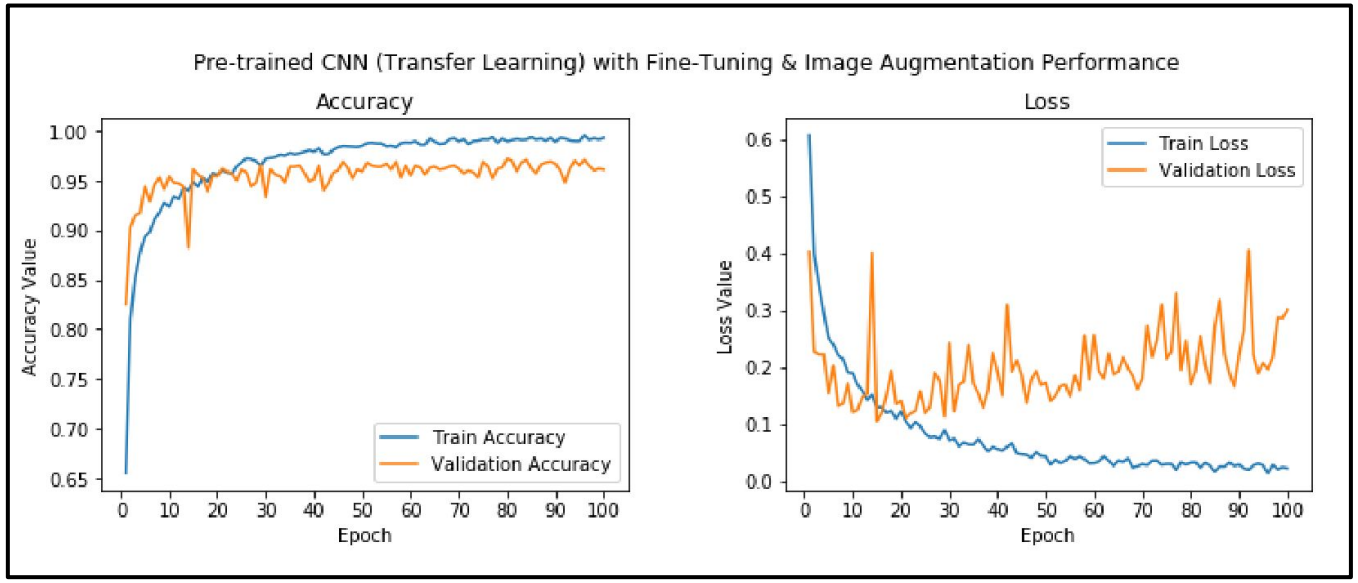


3.3 Improving Pretrained Model with fine-tuning and image augmentation

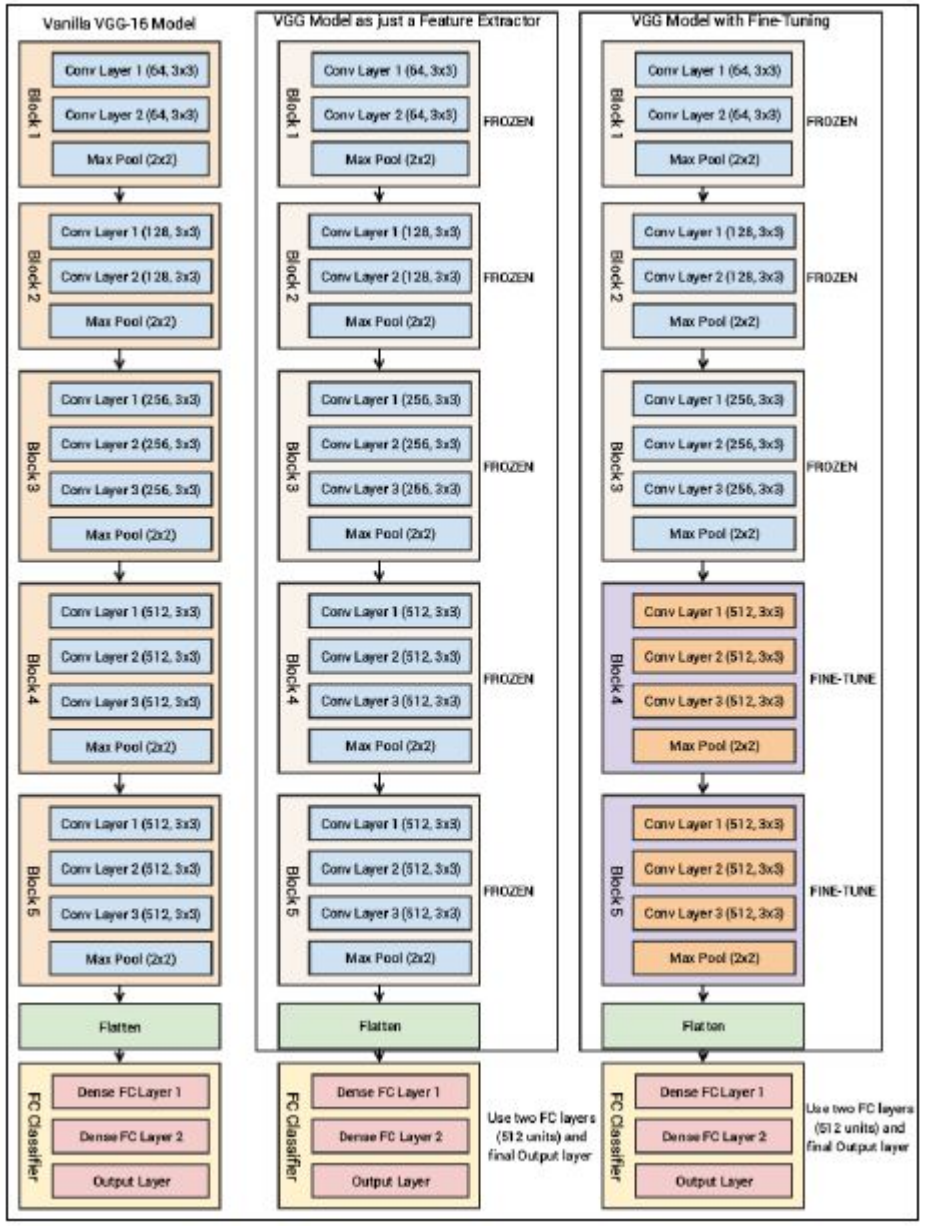
The pretrained VGG16 model with fine-tuning and image augmentation, where you unfreeze convolutional blocks 4 and 5 while keeping the first three blocks frozen

```
from keras.applications import VGG16
from keras.models import Model, Sequential
from keras.layers import Flatten, Dense, Dropout, InputLayer
from keras.optimizers import RMSprop
from keras.preprocessing.image import ImageDataGenerator
# Define input shape for your images and batch size
input_shape = (224, 224, 3)
batch_size = 32
epochs = 10  # Adjust as needed
 # Load pretrained VGG16 model without the top (fully connected) layers
base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
# Freeze layers from the first three blocks
for layer in base_model.layers[:15]:
    layer.trainable = False
# Unfreeze layers from block 4 and 5
for layer in base_model.layers[15:]:
    layer.trainable = True
```

```
# Data augmentation for training set
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)
# Data augmentation for validation set (only rescaling)
validation_datagen = ImageDataGenerator(rescale=1./255)
# Path to your training and validation data directories
train_dir = 'path_to_training_directory'
validation_dir = 'path_to_validation_directory'
# Flow training images in batches using train_datagen generator
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(input_shape[0], input_shape[1]),
    batch_size=batch_size,
    class_mode='binary'
)
# Flow validation images in batches using validation_datagen generator
validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=(input_shape[0], input_shape[1]),
    batch_size=batch_size,
    class_mode='binary'
)
```



```
# Create a new model using VGG16 base and custom top layers for classification
model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))

# Compile the model with binary crossentropy loss and RMSprop optimizer
model.compile(loss='binary_crossentropy',
              optimizer=RMSprop(lr=1e-4),
              metrics=['accuracy'])

# Print model summary
model.summary()
# Train the model using data generators
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.n // batch_size,
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=validation_generator.n // batch_size,
    verbose=1
)
```



Predict on a sample image

```
import numpy as np
from keras.preprocessing import image

# Path to your sample image
image_path = 'path_to_sample_image.jpg'

# Load and preprocess the image
img = image.load_img(image_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)  # Expand dimensions to create batch of size 1
img_array = img_array / 255.0  # Rescale pixel values to [0, 1], consistent with training data preprocessing

# Make predictions using the model
predictions = model.predict(img_array)

# Decode predictions (assuming binary classification)
predicted_class = 'Dog' if predictions[0][0] > 0.5 else 'Cat'  # Example threshold for binary classification

# Print the predicted class and probability
print(f"Predicted class: {predicted_class}")
print(f"Probability: {predictions[0][0]}")
```

3.4 Model Performance Evaluation on test data

```
from sklearn.metrics import classification_report
import numpy as np

# Assuming you have loaded your test data and labels using an ImageDataGenerator
test_generator = test_datagen.flow_from_directory(
    'path_to_test_directory',
    target_size=(224, 224),
    batch_size=batch_size,
    class_mode='binary',
    shuffle=False
)

# Evaluate the model on the test data
predictions = model.predict(test_generator)

# Convert predictions to binary classes (0 or 1)
predicted_classes = np.argmax(predictions, axis=1)

# Get true labels from the generator
true_classes = test_generator.classes

# Print classification report
print(classification_report(true_classes, predicted_classes, target_names=test_generator.class_indices))
```

```
Model Performance metrics:
-------------------------
Accuracy: 0.961
Precision: 0.9611
Recall: 0.961
F1 Score: 0.961
```

Model Classification report:

	precision	recall	f1-score	support
cat	0.97	0.95	0.96	500
dog	0.95	0.97	0.96	500
avg / total	0.96	0.96	0.96	1000

Prediction Confusion Matrix:

		Predicted: cat	Predicted: dog
Actual:	cat	476	24
	dog	15	485